that are defined separately. They are then added as tabs to a tab view which automatically translates the appearance of the tabs depending on the platform. This tabbed view itself will be added to the main view of the application.

This code will work on both Android and iOS — a slightly modified version is used for Windows Phone 7 — but will follow the native platform's conventions.

The same functionality is represented on each platform using the platform's native interface elements. While we showed C++ code for using these native widgets, it is also possible to use these widgets if you use the web programming route. There are two ways to go about this. You can either create widgets using a JavaScript API, or declaratively create the UI using HTML. The following is an example of creating a UI with JavaScript:

```
var myButton = mosync.nativeui.create("Button", "myButton", {
        "text" : "Click Me!",
        "width" : "100%"
});
```

Even if this widget is definitely declared in HTML, it will be translated to native code and will look different depending on platform.

The disadvantage when it comes to using the NativeUI is that it is only supported on the three main platforms, iOS, Android and Windows Phone 7. For other platforms, there is the option to use MoSync's alternative widget framework called MAUI, which includes non-native widgets that look the same on all platforms. An application can use Native widgets for the platform that support it, and non-native ones for the platforms that don't.

MAUI and MoRe

For platforms such as Symbian, Moblin, JavaME and Windows Mobile, you do not have the option to use the NativeUI widget framework. There instead you can use MAUI, a platform-independent UI framework included in MoSync.

A small example of how one might create a button using MAUI:

```
MAUI::WidgetSkin* buttonSkin = new MAUI::WidgetSkin( SELECTED_IMAGE, UNSELECTED_IMAGE, X1, X2, Y1, Y2, SELECTED_TRANSPARENT, UNSELECTED_TRANSPARENT);
myButton = new MAUI::Label(LEFT, TOP, WIDTH, HEIGHT, parentWidget, "Click Me!", COLOUR, FONT);
myButton->setSkin(buttonSkin);
```

Here we are first describing a skin for the button, which includes the images for the button when selected and unselected, information about how the image fits on the button, and whether the images are transparent. Then we are defining the button itself, and finally we are applying the button skin to it. This widget will look the same on all platforms.

MoSync supports emulating applications the emulators that ship with their respective SDKs, the Android emulator, the Windows Phone 7 emulator and the iOS simulator for example. However it also ships with an application called MoRE, which is a simulator for MoSync applications. Unlike device emulators, which have multiple layers, first emulating the device hardware, and then running the OS on top of that, MoRE simply runs the applications directly, so it starts quickly and performs better. However MoRE cannot be use with applications that use the NativeUI and some other features.

Web UI and Wormhole

MAUI isn't the only way to have a mobile application look consistently and work across all platforms. The other option is using web technologies as the frontend of your application.

Using standard web technologies such as HTML, CSS, and JavaScript one can create powerful UIs. As mentioned before, MoSync exposes native platforms features such as the camera, compass etc. to web applications so they are not feature-restricted.

We also talked about how it goes a step ahead by allowing one to mix and match native and web code. MoSync includes a technology called Wormhole that allows you to bridge between native C / C++ code and JavaScript web code. The C++ Wormhole library allows one to write C++ code that can be invoked from JavaScript. For example, one could write an image filter or an audio processing algorithm in C++ which could be used by the JavaScript code.

Conclusion

While we have covered different approaches to creating the UI, Native UI, MAUI and web-based UI, you needn't use just one. You can mix and match as desired.

As you can see MoSync is a powerful SDK and should be in the radar of anyone looking to create applications that need to run on multiple platforms. It is very flexible, with a broad feature set that is supported on an even broader set of devices.

Not all features are available on all platforms, that is true; however, it still promotes a large amount of code-reuse.

All of this of this is possible in MoSync via a single SDK. MoSync even offers an Eclipse-based IDE for working with their cross-platform SDK, supports testing using emulators for iOS, Android and Windows Phone, and includes a simulator for quickly testing apps. Oh, and did we mention that all this — the SDK and the IDE — is free and open source under the GPL. >

*pointers

>>Nuggets of cool code at work

*Android application development for beginners!

>>For those of you who requested tutorials, here's a series that begins with the bare basics! > 04:55

http://dvwx.in/may-12-p1

*Game programming with GWT

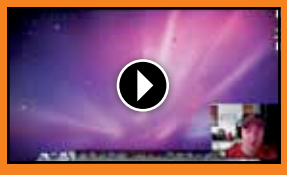
>>Learn to write 2D and 3D games using HTML5 and GWT, leverage and port existing games, and publish to the Chrome Web Store. > 44:59

http://dvwx.in/may-12-p2

*Windows 8 and HTML5

>>With a swipe of your finger, you may be able to launch Windows programs and navigate through the vast majority of new Web-based apps. > 03:02

http://dvwx.in/may-12-p3



*Objective-C tutorial

>>If you want to develop for the most popular app platform out there, you need to be familiar with Objective-C. Be it the iPod Touch, iPhone or iPad, Objective-C is your way to go. Here's a tutorial for absolute beginners.

Duration > 06:27

http://dvwx.in/may-12-p4